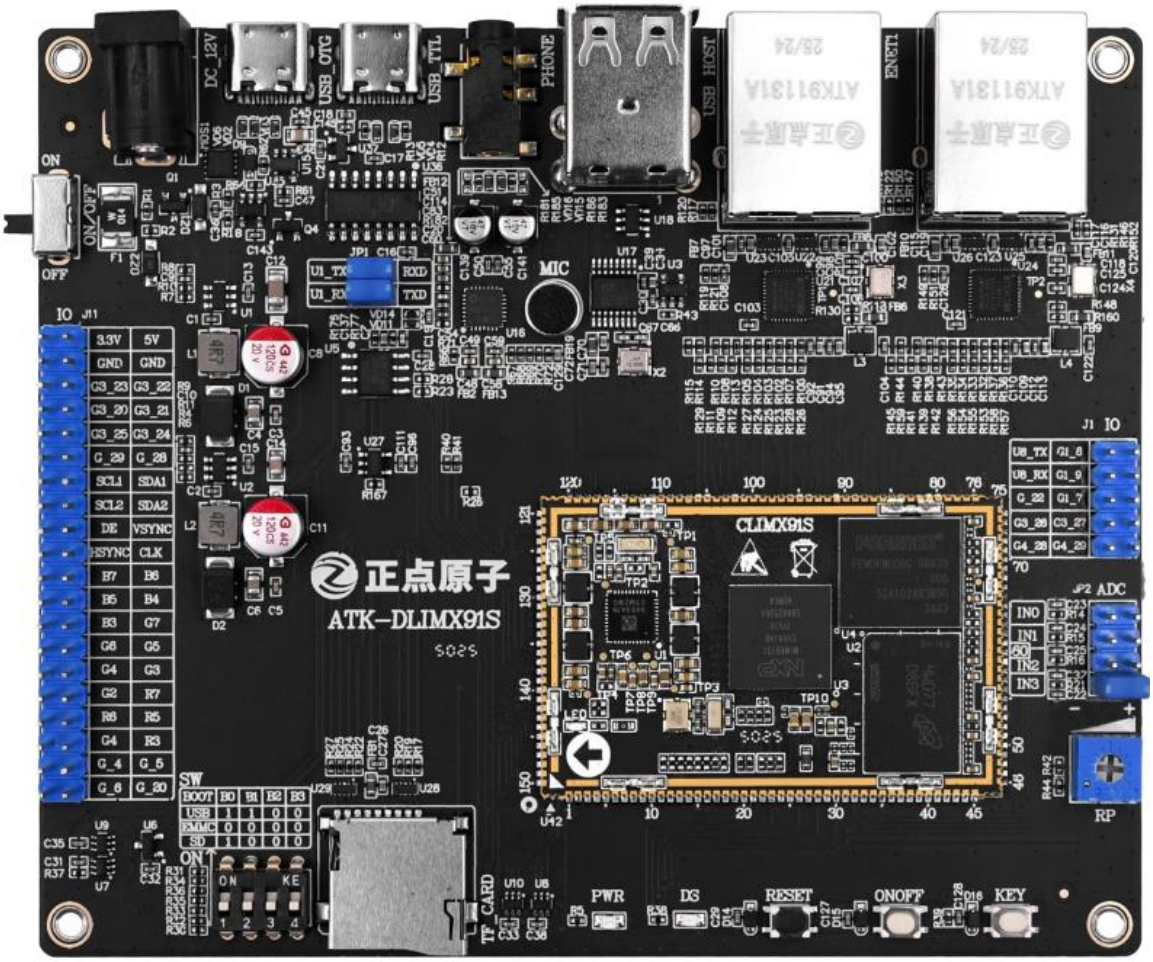
ATK-DLIMX91S

Factory System SDK User Manual

V1.0



ALIENTEK

1. Shopping:

TMALL: https://zhengdianyuanzi.tmall.com

TAOBAO: https://openedv.taobao.com

2. Download

Address: http://www.openedv.com/docs/index.html

3. FAE

Website : www.alientek.com

Forum : http://www.openedv.com/forum.php

Videos : www.yuanzige.com

Fax : +86 - 20 - 36773971

Phone : +86 - 20 - 38271790

Disclaimer

The product specifications and instructions mentioned in this document are for reference only and subject to update without prior notice; Unless otherwise agreed, this document is intended as a product guide only, and none of the representations made herein constitutes a warranty of any kind. The copyright of this document belongs to Guangzhou Xingyi Electronic Technology Co., LTD. Without the written permission of the company, any unit or individual shall not be used for profit-making purposes in any way of dissemination.

In order to get the latest version of product information, please regularly visit the download center or contact the customer service of Taobao ALIENTEK flagship store. Thank you for your tolerance and support.

Revision History:

Version	Version Update Notes	Responsible person	Proofreading	Date
V1.0	release officially	ALIENTEK	ALIENTEK	2026.04.1

Catalogue

Chapter 1. Environmental Description 1
1.1 Software Version 1
Chapter 2. Compilation of Factory System Source Code and Image Construction 2
2.1 Compiling U-Boot Source Code and Generating flash.bin Firmware 2
2.2 Compiling Factory Kernel Source Code and Modules 5
2.3 Compile the Factory QT GUI Comprehensive Interface 9

Chapter 1. Environmental Description

This document mainly describes the compilation process of the factory system source code for the ALIENTEK ATK-DLIMX91S development board. It guides users to master the compilation method of the factory source code, facilitating subsequent secondary development. This document does not explain the compilation principles of each module.

Operating environment used in this document:

- Windows 10 64-bit. It is not recommended to carry out development on 32-bit Windows systems, as 32-bit Windows has limited supported memory capacity and system performance. This document is described based on a 64-bit operating system.
- Ubuntu 20.04. Ubuntu 20.04 is recommended; otherwise, errors may occur due to inconsistent operating environments.
- Users are required to be proficient in using FileZilla or WinSCP to transfer files between Ubuntu and Windows.
- This document is compiled for the default factory system source code of the ALIENTEK ATK-DLIMX91S development board. If users adopt other source code versions or cross-compilers, compilation failures may be caused by differences in source code configuration, instruction set support of compilers and other factors. Such problems need to be solved independently, and the compilation commands in this document are for reference only. For the convenience of development, please use the official default factory system source code.

1.1 Software Version

Source Code / Tool	Version	Remarks
U-Boot and SPL	2024.04	
Linux kernel	6.6.52	
Qt	5.15.0	
aarch64-poky-linux-gcc	13.3.0	Cross-compilation Toolchain
UUU	1.5.201	NXP Programming Tool
Development Environment	Ubuntu20.04	Virtual Machine Image Provided by ALIENTEK

Chapter 2. Compilation of Factory System Source Code and Image Construction

The factory system source code is developed and modified by ALIENTEK based on the original source code provided by NXP, in order to adapt to the hardware resources of the ATK-DLIMX91S development board. Users can carry out secondary development and debugging based on this factory system source code. For example, if the self-designed baseboard has different hardware resources from the development board, you can modify part of the factory system source code and compile and debug it directly.

The factory system source code is stored in the cloud disk at the following path:

IMX91 Development Board\Development Board Disc A - Basic Documents\01_Program Source Code\01_ALIENTEK Linux Factory System Source Code

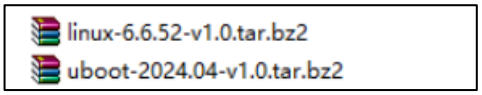


Figure 1.1-1 Factory System Source Code Example

As shown in the figure above, the factory source code will be updated with version iterations when necessary, and information such as the version number may differ from the example in the figure. Users can use it directly.

To facilitate learning driver development, the Linux kernel source code is not managed with Git and only retains one release version number. For the convenience of explanation in this document, version **v1.0** is uniformly used as an example. The actual source code version shall be subject to the latest release. Please modify the source code extraction commands involving the source code version below accordingly.

The compilation methods for each part of the source code are introduced below.

Before compilation, please create a directory named **ATK-DLIMX91S** in any directory under the Ubuntu environment. All subsequent compilation operations of the factory source code will be performed in this ATK-DLIMX91S directory.

In addition, the following tools must be installed in the Ubuntu 20.04 environment to avoid missing tool library prompts during compilation.

```
sudo apt-get install make gcc libssl-dev g++ git libncurses5-dev libncursesw5-dev libyaml-dev
sudo apt-get install u-boot-tools python3-pyelftools device-tree-compiler bison flex expect
```

2.1 Compiling U-Boot Source Code and Generating flash.bin Firmware

The binary files for both U-Boot and SPL (u-boot.bin and u-boot-spl.bin) can be compiled simultaneously from the U-Boot source code.

Before compiling U-Boot, the required libraries must be installed by entering the following commands:

```
sudo apt-get install bison flex
```

Create a directory named **alientek_uboot** under the **ATK-DLIMX91S** directory in Ubuntu, then copy the ALIENTEK factory U-Boot source package uboot-2024.04-v1.0.tar.bz2 into the alientek_uboot directory.

The decompression command is as follows: (Please confirm the source code version you have downloaded; do not directly copy the decompression command below.)

```
tar -xjf uboot-2024.04-v1.0.tar.bz2
```

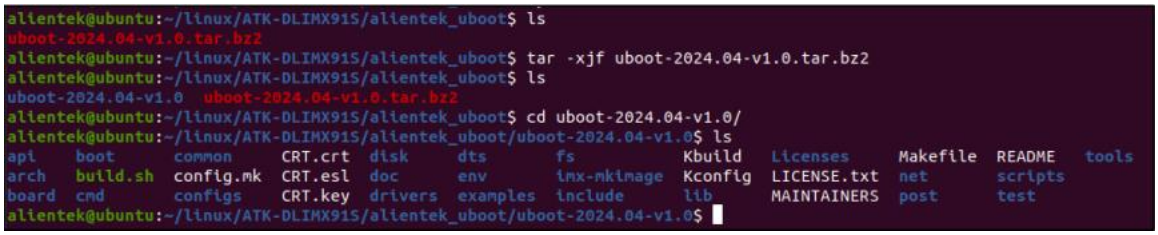


Figure 2.1-1 U-Boot Source Code After Extraction

When compiling the factory source code, users do not need to modify anything. To facilitate compilation, ALIENTEK provides a one-click compilation script named build.sh. You can run the script directly for compilation the first time, as shown below:

```
if [ ! -e "/opt/fsl-imx-xwayland/6.6-scarthgap/environment-setup-armv8a-poky-linux" ]; then echo ""
echo " Please first install the cross-compiler from the Disc A - Basic Documentation of the ALIENTEK ATK-DLIMX91 Development Board -> 05. Development Tools -> 01. Cross Compiler.
->fsl-imx-xwayland-glibc-x86_64-imx-image-core-armv8a-imx91-11x11-lpddr4-evk-toolchain-6.6-scarthgap.sh"
echo "" fi
# Use the GCC 13.3.0 cross-compiler from the Yocto SDK to compile the factory Linux source code. There is no need to specify ARCH and other parameters; you can directly run the make command.
source /opt/fsl-imx-xwayland/6.6-scarthgap/environment-setup-armv8a-poky-linux
#!/bin/bash # Clean before compiling.
make distclean
make imx91_11x11_atk_defconfig make all -j16
# Create a new tmp directory in the current directory to store the compiled target files.
if [ ! -e "./tmp" ]; then mkdir tmp
fi
rm -rf tmp/*
# Copy the compiled u-boot.bin and u-boot-spl.bin to the current tmp directory.
mv u-boot.bin tmp mv spl/u-boot-spl.bin tmp
# Compile the flash.bin file. #clean imx-mkimage
# Check if the file exists and delete it.
echo " Enter imx-mkimage/iMX91" cd imx-mkimage/iMX91
delete_if_exist() { if [ -e "$1" ]; then
echo " Delete the file: $1" rm -f "$1"
fi }
# List the files that need to be deleted.
files_to_delete=( "flash.bin" "head.hash" "u-boot.bin"
"u-boot-spl.bin" "u-boot-hash.bin"
"u-boot-spl-ddr.bin" "boot-spl-container.img"
"u-boot-atf-container.img" )
```

```
# Traverse the file list and perform deletion operations.
for file in "${files_to_delete[@]}"; do delete_if_exist "$file"
done sync sync
echo " Copy u-boot.bin and u-boot-spl.bin from tmp to imx-mkimage/iMX9" cd -
cp tmp/u-boot.bin tmp/u-boot-spl.bin imx-mkimage/iMX91
sync
echo " Enter imx-mkimage and compile flash.bin" cd imx-mkimage
make SOC=iMX91 REV=A0 flash_singleboot
sync
sync
cp iMX91/flash.bin ../tmp
echo " Compilation completed. Please check the tmp folder in the current directory for the compiled target files."
```

This script includes operations such as enabling the cross-compiler, compiling the U-Boot source code, and packaging flash.bin. Execute the build.sh script.

```
./build.sh
```

Upon successful compilation, the relevant files will be stored in the tmp directory, as shown in the figure below:

```
container image offset (aligned):19800
csf_off         0x19c00
flags: 0x10
Hash of the images =
1+0 records in
1+0 records out
190464 bytes (190 kB, 186 KiB) copied, 0.000378597 s, 503 MB/s
370+1 records in
370+1 records out
189576 bytes (190 kB, 185 KiB) copied, 0.000869025 s, 218 MB/s
AP file_offset = 0x19800 size = 0x2e800
CST: CONTAINER 0 offset: 0x400
CST: CONTAINER 0: Signature Block: offset is at 0x490
        Offsets =       0x400   0x490
DONE.
Note: Please copy image to offset: IVT_OFFSET + IMAGE_OFFSET
append u-boot-atf-container.img at 288 KB, psize=1024
1659+0 records in
1659+0 records out
1698816 bytes (1.7 MB, 1.6 MiB) copied, 0.00554745 s, 306 MB/s
编译完成，请查看当前目录下的tmp文件夹查看编译好的目标文件
```

Figure 2.1-2 Compilation Completed

The flash.bin file is the firmware that can be burned to the development board. This firmware is generated by the imx-mkimage tool and consists of files including uboot.bin, u-boot-spl.bin, bl31.bin, and tee.bin.

After compiling flash.bin, users can use the UUU programming tool and the uuu-flash programming script to burn and update flash.bin to the development board. Set the development board DIP switches to USB programming mode, connect the USB_OTG and USB_TTL interfaces, mount the development board OTG to the virtual machine, and open the USB_TTL serial port to view the detailed programming information. The programming commands are as follows:

```
cd uboot-2024.04-v1.0/imx-mkimage/iMX91
sudo uuu uuu-flash
```

```
alientek@ubuntu:~/linux/ATK-DLIMX91S/alientek_uboot/uboot-2024.04-v1.0/imx-mkimage/iMX91$ sudo uuu uuu-flash
[sudo] password for alientek:
uuu (Universal Update Utility) for nxp imx chips -- libuuu_1.5.201-0-g727fc2b

Success 1    Failure 0


3:2-6FB2D253 1/ 1 [=================100%=================] SDPS: boot -f flash.bin
3:41-6FB2D25 7/ 7 [Done                                   ] FB: done
```

Figure 2.1-3 Programming flash.bin Using uuu-flash

At this point, flash.bin has been updated to the development board. If you do not want to use UUU, you can copy flash.bin to the development board for firmware update. For detailed firmware update operations, please refer to the document in the development board Disc A - Basic Data:

\10. User Manual\【ALIENTEK】ATK-DLIMX91S Firmware Update Reference.pdf

Notes:

(1) If the error **"uuu: command not found"** occurs, it indicates that the system cannot recognize the uuu command. The problem can be solved by performing the following operations.

```
sudo apt-get install libtinyxml2-dev
sudo cp uboot-2024.04-v1.0/imx-mkimage/iMX91/uuu /usr/bin
sudo chmod 777 /usr/bin/uuu
```

(2) If the following GLIBC-related error occurs:

```
./../mkimage_imx8: /lib/x86_64-linux-gnu/libc.so.6: version GLIBC_2.33' not found (required by ./../mkimage_imx8)
./../mkimage_imx8: /lib/x86_64-linux-gnu/libc.so.6: version GLIBC_2.34' not found (required by ./../mkimage_imx8)
./../mkimage_imx8: /lib/x86_64-linux-gnu/libc.so.6: version GLIBC_2.38' not found (required by ./../mkimage_imx8)
make[1]: *** [soc.mak:96: u-boot-hash.bin] Error 1
make: *** [Makefile:26: flash_singleboot] Error 2
```

You need to delete the mkimage_imx8 folder under the imx-mkimage directory, and then re-run the build.sh script in the U-Boot source code.

```
rm -f uboot-2024.04-v1.0/imx-mkimage/mkimage_imx8
./build.sh
```

2.2 Compiling Factory Kernel Source Code and Modules

Create a directory named **alientek_linux** under the **ATK-DLIMX91S** directory in Ubuntu, then copy the ALIENTEK factory Linux source package linux.6.6.52-v1.0.tar.bz2 into the alientek_linux directory. Run the decompression command and enter the kernel source directory. (Please confirm the source code version you have downloaded; do not directly copy the decompression command below.)

```
tar -xjf linux-6.6.52-v1.0.tar.bz2
cd linux-6.6.52-v1.0
```

The running result is shown in the figure below:

```
alientek@ubuntu:~/linux/ATK-DLIMX91S/alientek_linux$ ls
linux-6.6.52-v1.0.tar.bz2
alientek@ubuntu:~/linux/ATK-DLIMX91S/alientek_linux$ tar -xjf linux-6.6.52-v1.0.tar.bz2
alientek@ubuntu:~/linux/ATK-DLIMX91S/alientek_linux$ ls
linux-6.6.52-v1.0  linux-6.6.52-v1.0.tar.bz2
alientek@ubuntu:~/linux/ATK-DLIMX91S/alientek_linux$ cd linux-6.6.52-v1.0/
alientek@ubuntu:~/linux/ATK-DLIMX91S/alientek_linux/linux-6.6.52-v1.0$ ls
arch      COPYING        drivers  io_uring  kernel       MAINTAINERS.NXP  README   security  virt
block     CREDITS        fs       ipc       lib          Makefile         rust     sound
build.sh  crypto         include  Kbuild    LICENSES     mm               samples  tools
certs     Documentation  init     Kconfig   MAINTAINERS  net              scripts  usr
alientek@ubuntu:~/linux/ATK-DLIMX91S/alientek_linux/linux-6.6.52-v1.0$
```

Figure 2.2-1 Linux Source Code After Extraction

When compiling the factory source code, users do not need to modify anything.

The device tree path of the Linux kernel source code adapted to the hardware resources of the ATK-DLIMX91S development board is:

arch/arm64/boot/dts/freescale/dts/

The device tree file is imx91-11x11-atk.dts.

To facilitate compilation, ALIENTEK provides a one-click compilation script build.sh. You can run the script directly for compilation the first time, as shown below:

```
if [ ! -e "/opt/fsl-imx-xwayland/6.6-scarthgap/environment-setup-armv8a-poky-linux" ]; then
echo ""
echo "Please install the cross-compiler first from the ALIENTEK ATK-DLIMX91 Development Board Disc A - Basic Documents -> 05. Development Tools -> 01. Cross Compiler"
echo "-> fsl-imx-xwayland-glibc-x86_64-imx-image-multimedia-armv8a-imx91-11x11-lpddr4-evk-toolchain-6.6-scarthgap.sh"
echo ""
fi

# Compile the factory Linux source code using the GCC 13.3.0 cross-compiler from the Yocto SDK.# There is no need to specify ARCH and other parameters; simply execute make directly.
source /opt/fsl-imx-xwayland/6.6-scarthgap/environment-setup-armv8a-poky-linux

#!/bin/bash
# Clean up before compilation
#make clean

# Configure the defconfig file
make imx_v8_defconfig -j 16

# Start compiling Image
#make menuconfig
make Image -j 16

# Compile device trees for various ALIENTEK display devices.
  # If no display is connected, imx91-11x11-atk.dtb will be loaded by default at startup.
make freescale/imx91-11x11-atk.dtb

```

```
25 # Compile kernel modules
26 make modules -j16
27
28 # Create a new tmp directory in the current directory to store compiled target files
29 if [ ! -e "./tmp" ]; then
30 mkdir tmp
31 fi
32 rm -rf tmp/*
33
34 # Install compiled modules to the tmp directory,
   # use INSTALL_MOD_STRIP=1 to remove module debugging information
35 make modules_install INSTALL_MOD_PATH=tmp INSTALL_MOD_STRIP=1 -j16
36
37 # Delete the source directory under the module directory
38 rm -rf tmp/lib/modules/6.6.52/source
39
40 # Delete the build directory under the module directory
41 rm -rf tmp/lib/modules/6.6.52/build
42
43 # Enter the module directory
44 cd tmp/lib/modules
45
46 # Compress kernel modules
47 tar -jcvf ../../modules.tar.bz2 .
48 cd -
49 rm -rf tmp/lib
50
51 # Copy Image to the tmp directory
52 cp arch/arm64/boot/Image tmp
53
54 # Copy all compiled device tree files to the current tmp directory
55 cp arch/arm64/boot/dts/freescale/imx91-11x11-atk*.dtb tmp
56
57 echo "Compilation completed. Please check the compiled target files in the tmp folder under the current directory."
```

Lines 1 to 6 check if the corresponding cross-compilation toolchain is installed on the current system. If not, the script stops running and prompts the user.

Line 9 enables the cross-compilation toolchain.

Line 13 cleans all compilation outputs. Users can choose to comment out this command according to their own needs.

Line 16 selects the configuration file. Running this command generates a ".config" file in the Linux source code directory.

Line 19 launches the kernel menu configuration. It is commented out by default and not executed. It can be enabled when kernel configuration changes are needed. After modifying the kernel configuration, save it as ./arch/arm64/configs/imx_v8_defconfig.

Line 20 starts compiling the kernel.

Line 23 starts compiling the ALIENTEK device tree.

Line 26 starts compiling kernel modules.

Lines 29 to 32 create a tmp directory to store the required files generated after compilation.

Line 35 packages the compiled modules into the tmp directory.

Line 38 deletes the source directory.

Line 41 deletes the build directory.

Line 47 packages the kernel modules.

Line 52 copies the Image to the tmp directory.

Line 55 copies the required device trees for the ATK-DLIMX91S development board to the tmp directory.

The above script can be run as individual commands (a .config file is required), so you don't need to use the full script every time. For example, when the author needs to clean the kernel compilation, they can copy and run line 13 of the script. You can compile directly by running the script, and the code is shown below:

```
./build.sh
```

The compilation result is shown in the figure below:

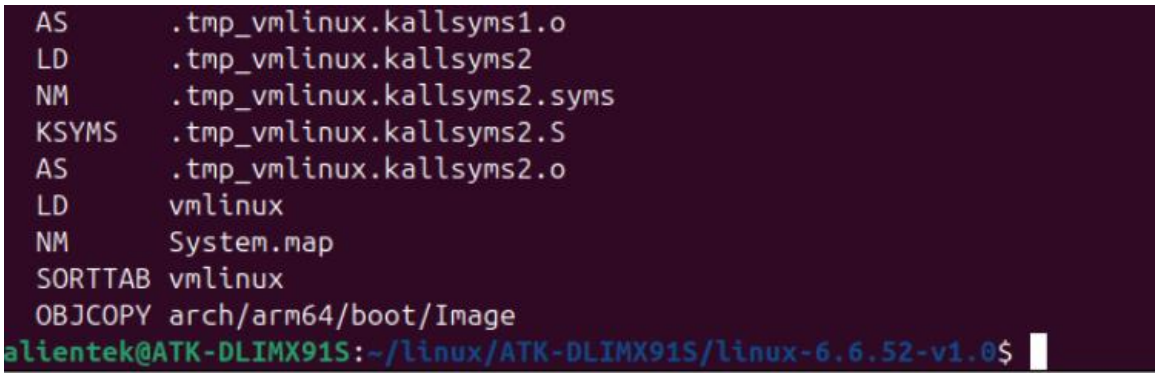


Figure 2.2-2 Linux Source Code Compilation Result

If you need to reconfigure the factory kernel, enter the following command to open the kernel menuconfig interface for configuration and selection.

```
make menuconfig
make savedefconfig
cp defconfig arch/arm64/configs/imx_v8_defconfig
```

After finishing modifying the kernel menu, we replace the original factory source code configuration file

arch/arm64/configs/imx_v8_defconfig

with the modified defconfig file.

We can then check the files in the tmp directory. The kernel Image and device tree files generated by compilation are as follows:

Image Name	Image Function	Image File and Path
Kernel	Linux Image	Image
Device Tree	Basic peripheral functions of the development board, such as Ethernet, TF card, etc.	imx91-11x11-atk.dtb
Kernel Modules	All modules that need to be loaded after the kernel starts are contained in this compressed package.	modules.tar.bz2

The factory system is compatible with various ALIENTEK LCD RGB screens, among which imx91-11x11-atk.dtb serves as the base device tree blob. When users need to modify the device tree, they should edit the source file imx91-11x11-atk.dts, compile the corresponding device tree file, and update it to the development board for use.

To verify the modified functions, you can copy the compiled files to the corresponding directories on the development board system. The Image and dtb files are located in /run/media/mmcblk0p1 on the factory system of the development board, and the kernel modules are located in /lib/modules/6.6.52.

For more firmware update operations, please refer to the document in the development board Disc A: Basic Data → 10. User Manual → *ALIENTEK ATK-DLIMX91S Firmware Update Reference Document.pdf*.

2.3 Compile the Factory QT GUI Comprehensive Interface

Please refer to the document in the development board Disc A - Basic Data:

\10. User Manual *ALIENTEK ATK-DLIMX91S Factory Interface QtUI User Manual.pdf*